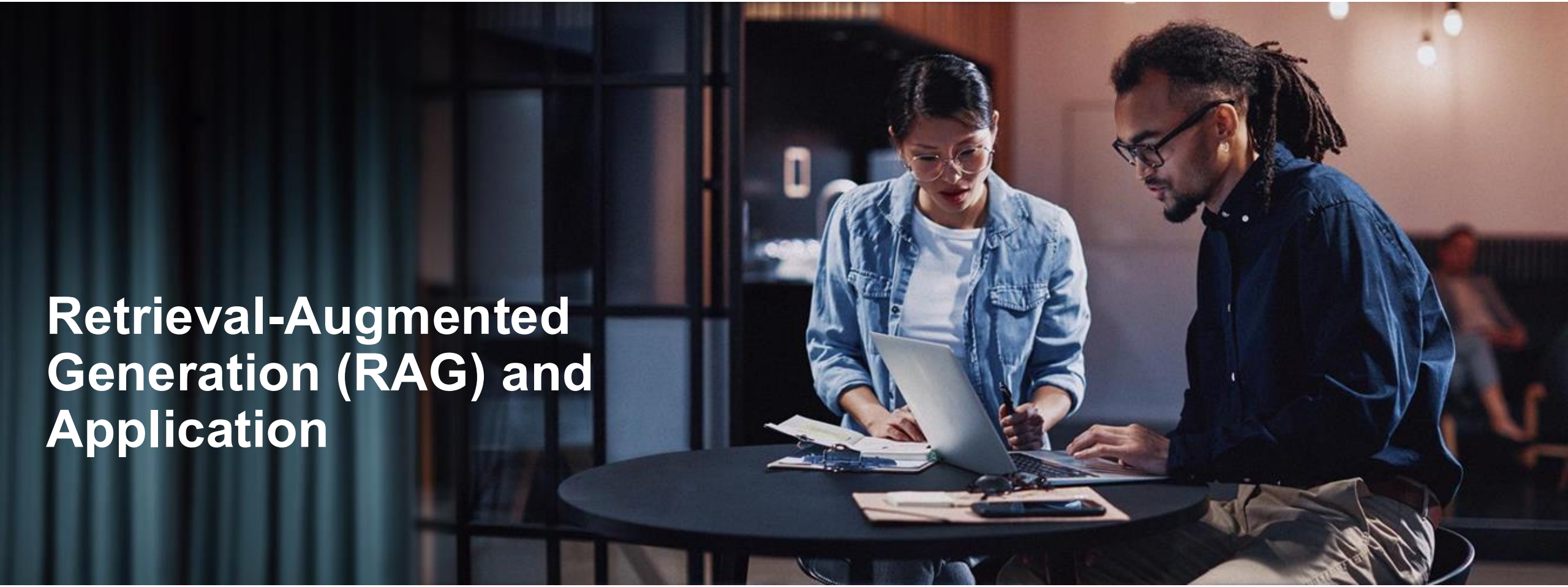
Retrieval-Augmented Generation (RAG) and Application



Gen AI Adoption



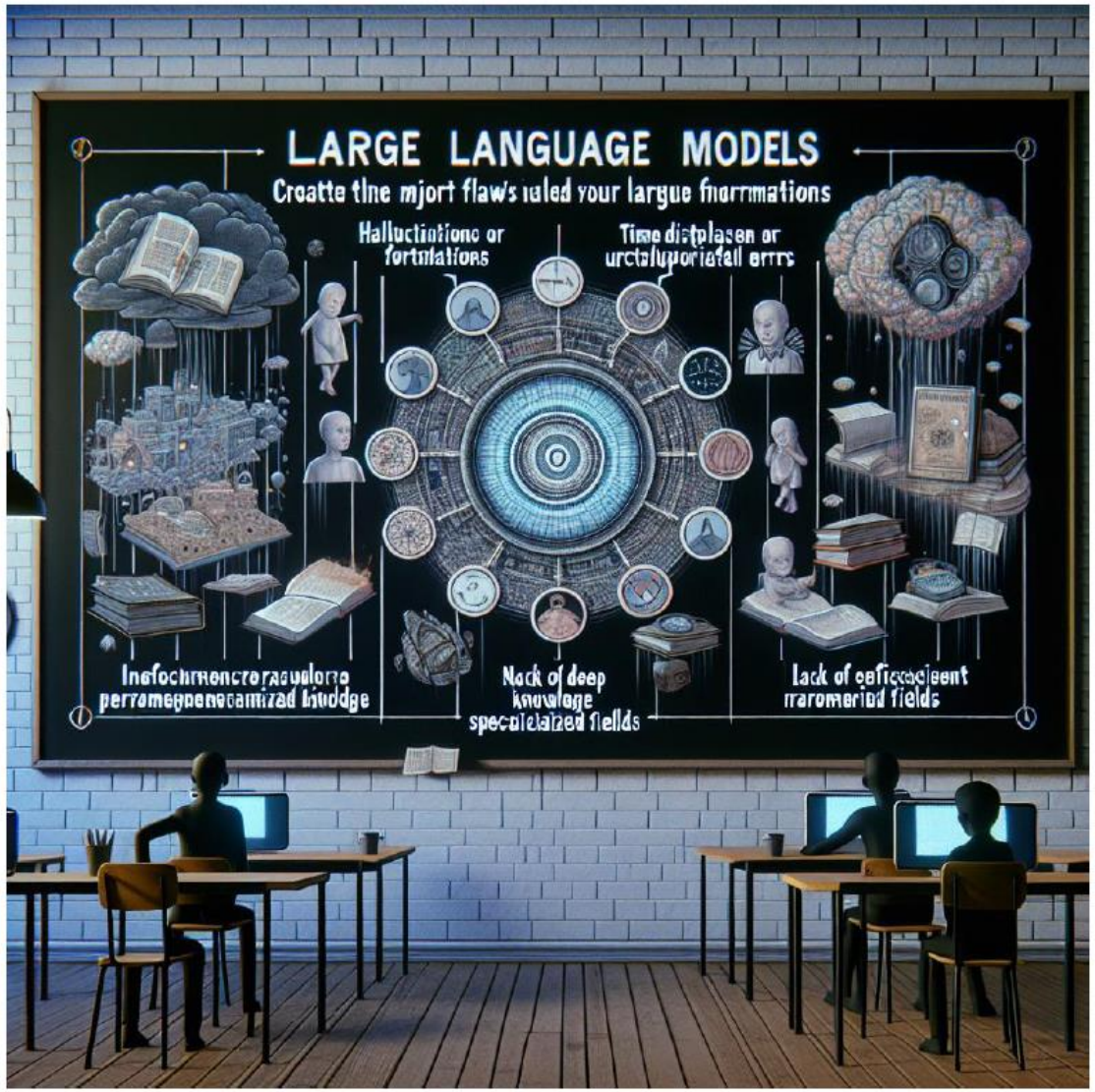
By 2026, more than 80% of enterprises will have used generative AI APIs and models and/or deployed GenAI-enabled applications in production environments, up from less than 5% in 2023.

Drawbacks of LLMs

- Hallucination
- Outdated information
- Low efficiency in parameterizing knowledge
- Lack of in-depth knowledge in specialized domains
- Weak inferential capabilities

Practical Requirements of Application

- Domain-specific accurate answering
- Frequent updates of data
- Traceability and explainability of generated content
- Controllable Cost
- Privacy protection of data



Draw by DALL·E-3

‘What’s the **capital of Wakanda**?’

‘What’s 5 times the **temperature of the sun**?’



Introduction to RAG

RAG (Retrieval-Augmented Generation) integrates data retrieval with language generation, enabling models to pull real-time, relevant information for more accurate responses.

Key Point: Combines the strengths of retrieval systems and generative AI.



...RAG

Retrieval:

- When an AI like ChatGPT doesn't know something, it usually guesses based on what it's been trained on. But RAG doesn't guess blindly.
- Instead, it searches external sources (like books, databases, or the internet) for real, up-to-date information.

Augmentation:

- The retrieved information is added to what the AI already knows. This makes the AI smarter, more accurate, and more reliable.

Generation:

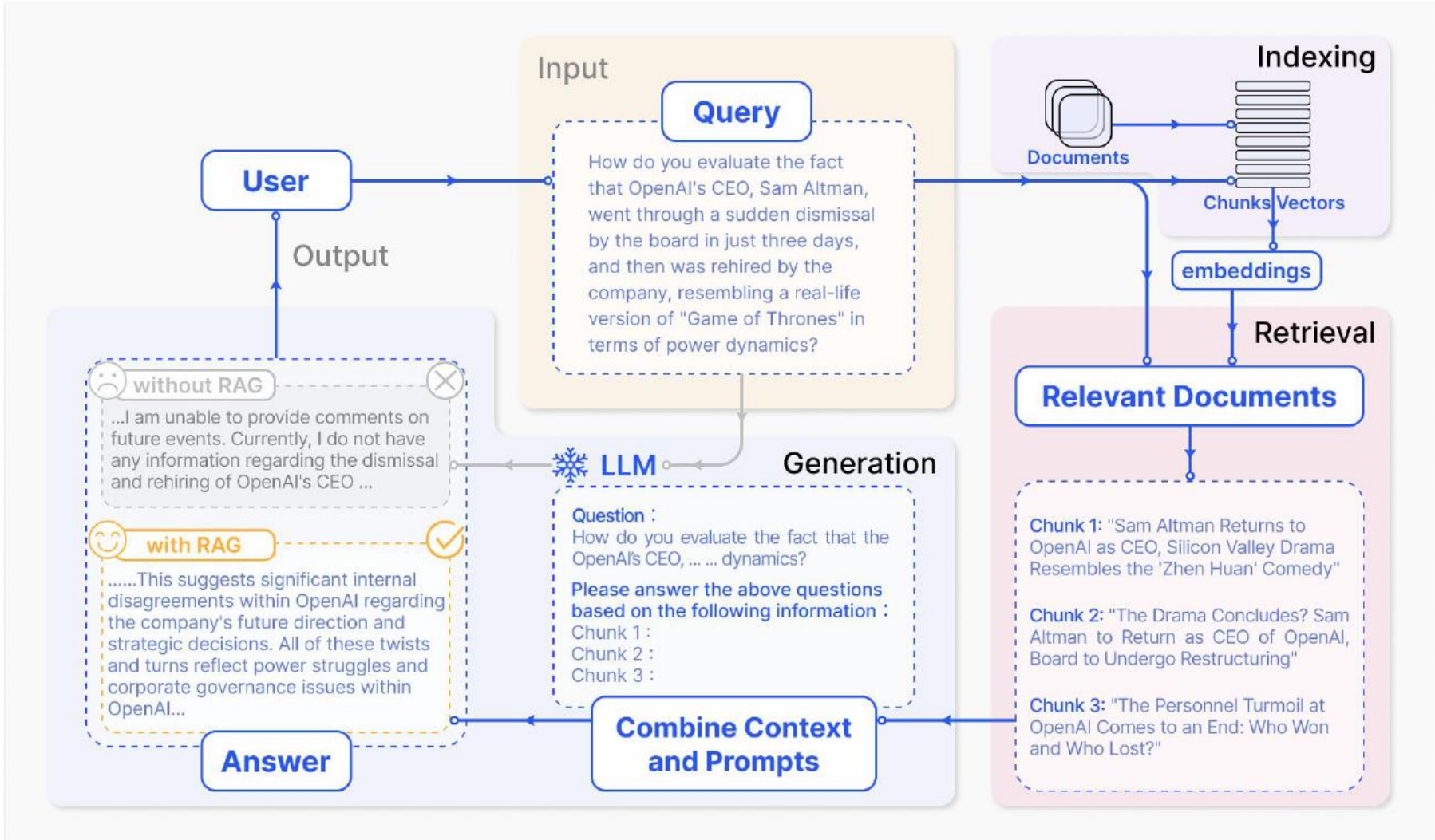
- Using its language skills, the AI takes the retrieved facts and writes a polished, human-like answer.

Retrieval-Augmented Generation (RAG)

When answering questions or generating text, it first retrieves relevant information from a large number of documents, and then LLMs generates answers based on this information.

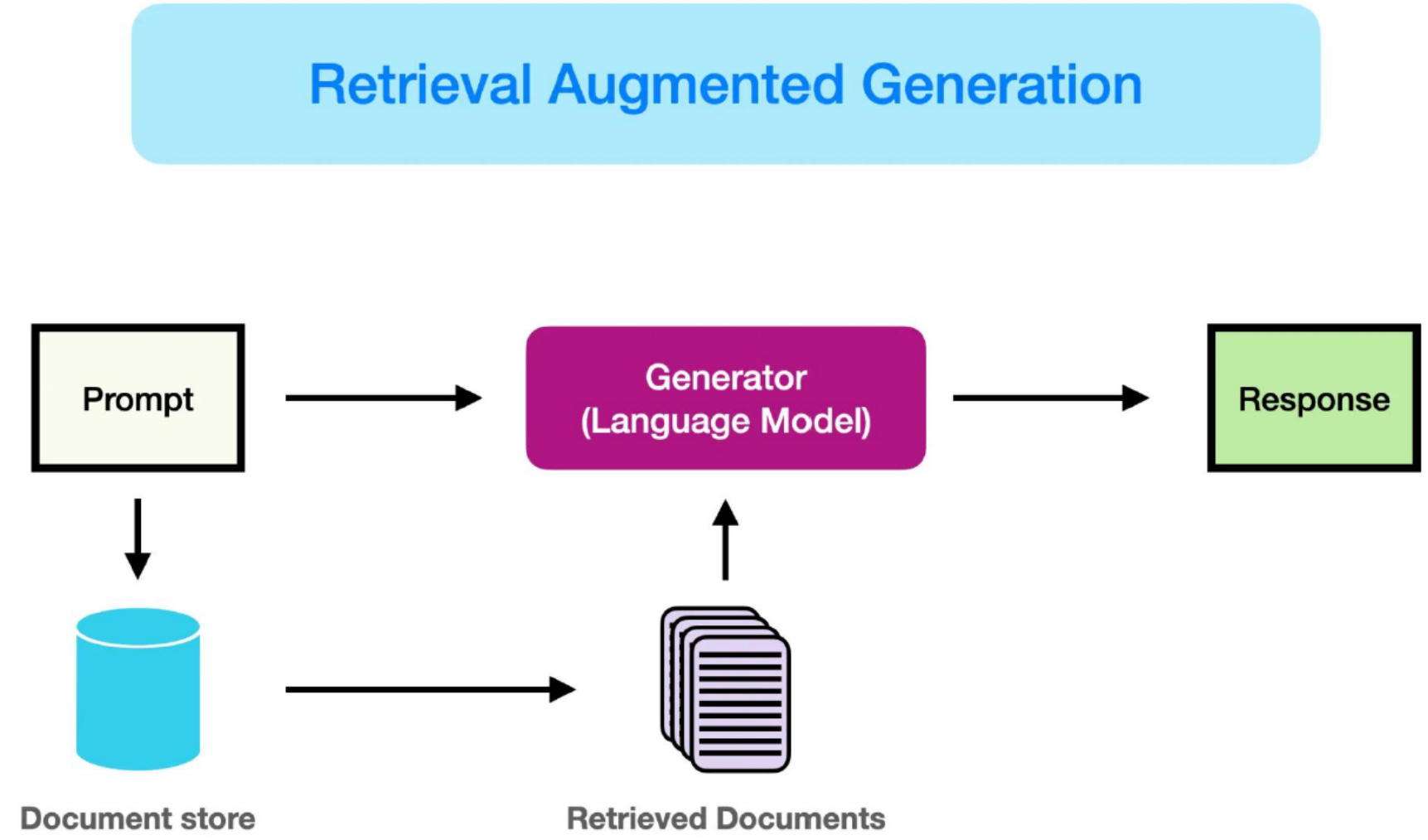
By attaching a external knowledge base, there is no need to retrain the entire large model for each specific task.

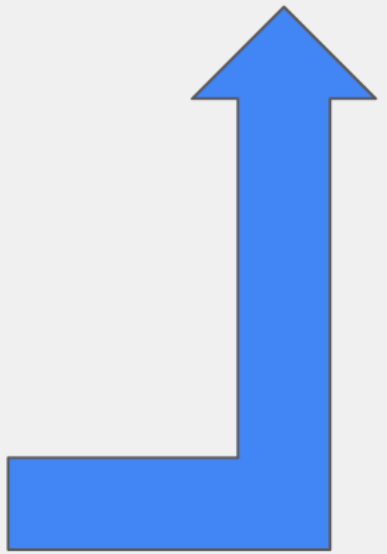
The RAG model is especially suitable for knowledge-intensive tasks.

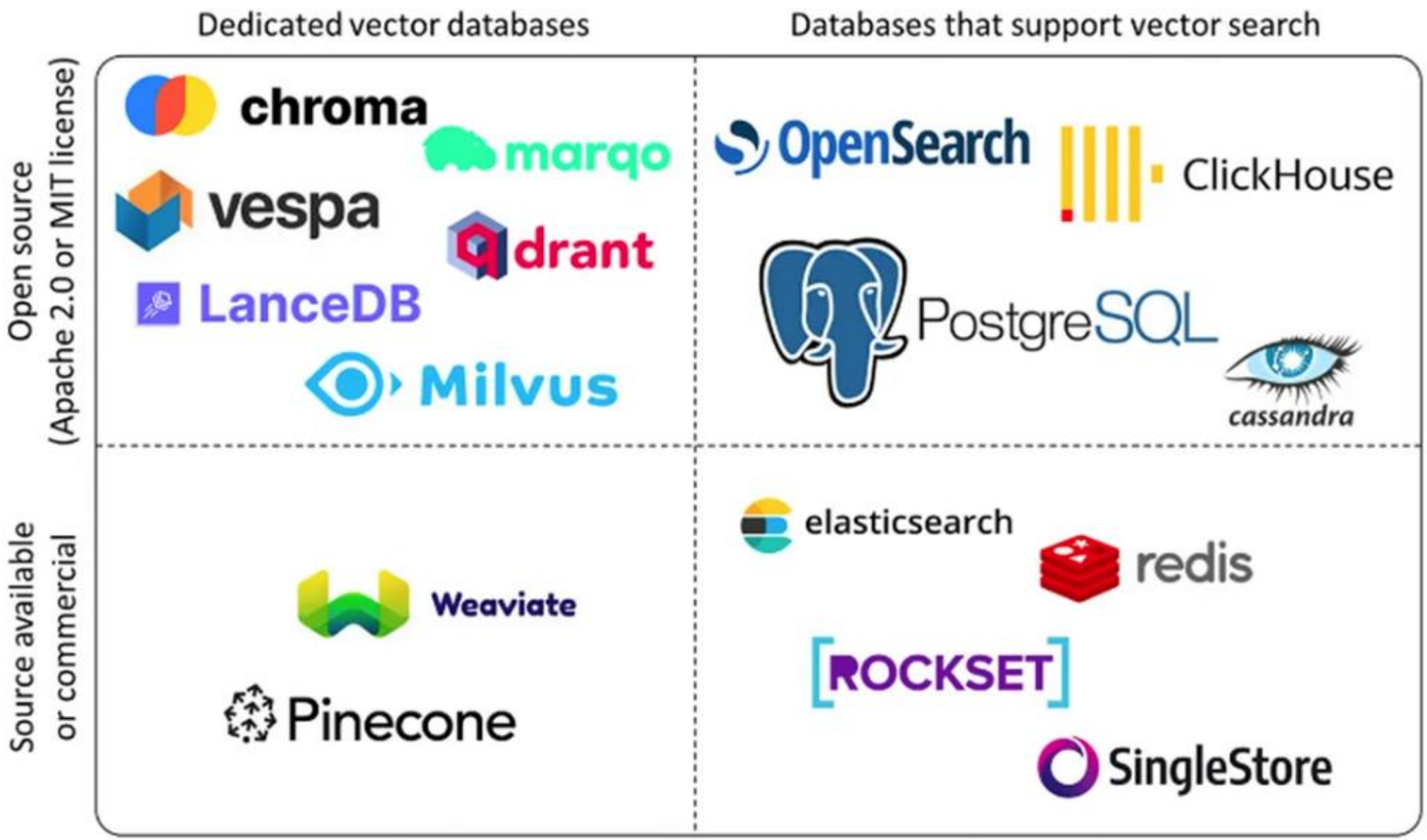


A typical case of RAG

RAG VISUALISED






Dedicated vector databases
Databases that support vector search
Open source (Apache 2.0 or MIT license)
Source available or commercial
chroma
marqo
vespa
qdrant
LanceDB
Milvus
OpenSearch
ClickHouse
PostgreSQL
cassandra
elasticsearch
redis
ROCKSET
SingleStore
Weaviate
Pinecone

Why RAG Matters

Rationale:

- Addresses the limitations of traditional language models constrained by static training data.
- Enhances response relevance and accuracy.
- Reduce **Hallucination** & Training Costs

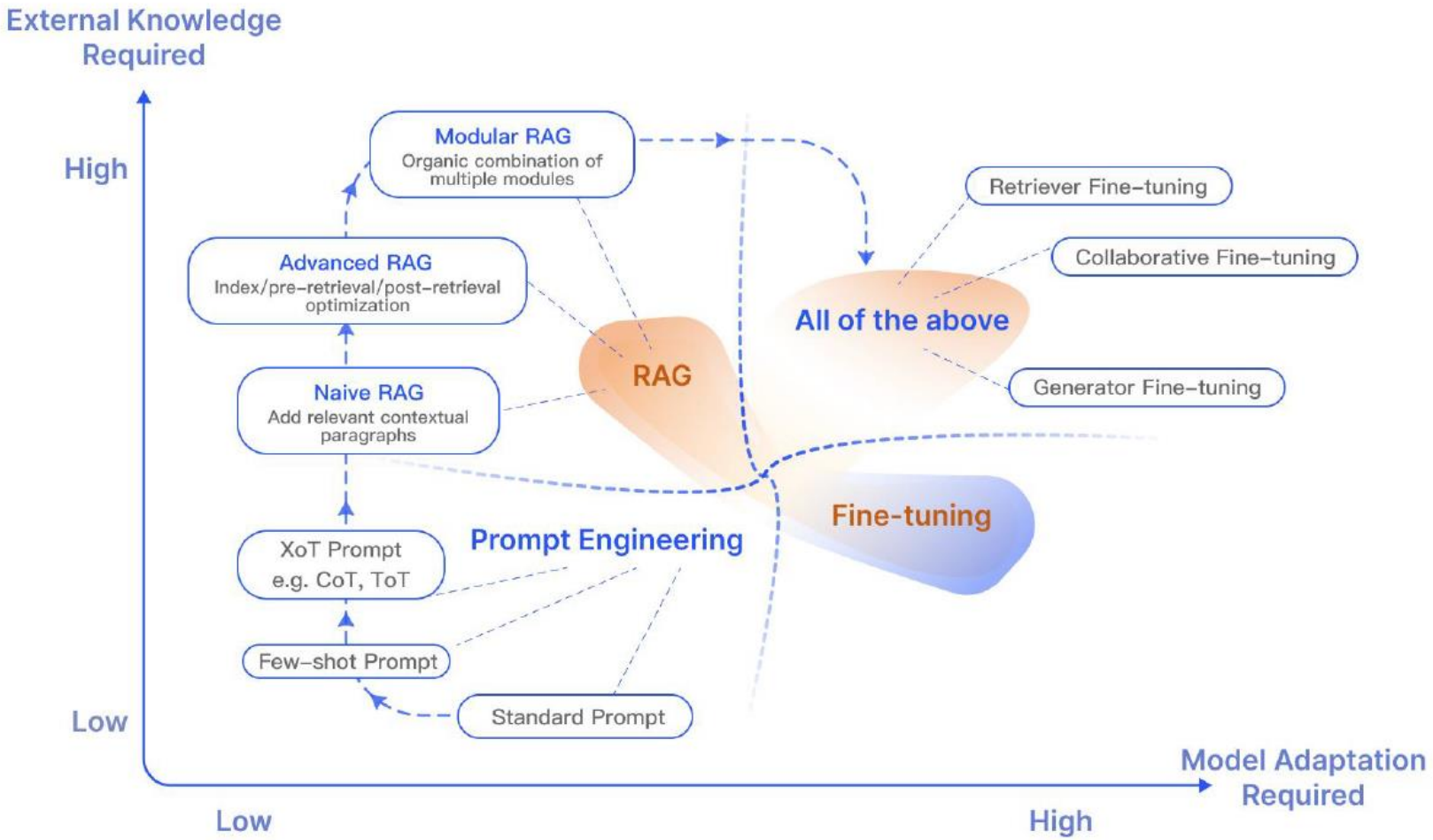
Modern Example:

- **ChatGPT with Browsing Capabilities**: Uses real-time web retrieval to answer questions that require up-to-date information.



Ways to optimize LLMs.

- Prompt Engineering
- Retrieval-Augmented Generation
- Instruct / Fine-tuning



A typical case of RAG

RAG vs Fine-tuning

Feature Comparison	RAG	Fine-Tuning
Knowledge Updates	Directly updating the retrieval knowledge base ensures that the information remains current without the need for frequent retraining, making it well-suited for dynamic data environments.	Stores static data, requiring retraining for knowledge and data updates.
External Knowledge	Proficient in leveraging external resources, particularly suitable for accessing documents or other structured/unstructured databases.	Can be utilized to align the externally acquired knowledge from pretraining with large language models, but may be less practical for frequently changing data sources.
Data Processing	Involves minimal data processing and handling.	Depends on the creation of high-quality datasets, and limited datasets may not result in significant performance improvements.
Model Customization	Focuses on information retrieval and integrating external knowledge but may not fully customize model behavior or writing style.	Allows adjustments of LLM behavior, writing style, or specific domain knowledge based on specific tones or terms.
Interpretability	Responses can be traced back to specific data sources, providing higher interpretability and traceability.	Similar to a black box, it is not always clear why the model reacts a certain way, resulting in relatively lower interpretability.
Computational Resources	Depends on computational resources to support retrieval strategies and technologies related to databases. Additionally, it requires the maintenance of external data source integration and updates.	The preparation and curation of high-quality training datasets, defining fine-tuning objectives, and providing corresponding computational resources are necessary.
Latency Requirements	Involves data retrieval, which may lead to higher latency.	LLM after fine-tuning can respond without retrieval, resulting in lower latency.
Reducing Hallucinations	Inherently less prone to hallucinations as each answer is grounded in retrieved evidence.	Can help reduce hallucinations by training the model based on specific domain data but may still exhibit hallucinations when faced with unfamiliar input.
Ethical and Privacy Issues	Ethical and privacy concerns arise from the storage and retrieval of text from external databases.	Ethical and privacy concerns may arise due to sensitive content in the training data.

RAG Applications

Scenarios where RAG is applicable:

- Long-tail distribution of data
- Frequent knowledge updates
- Answers requiring verification and traceability
- Specialized domain knowledge
- Data privacy preservation

Q&A
RETRO (Borgeaud et al2021)
REALM (Gu et al, 2020)
ATLAS (lzacard et al, 2023)

Fact Checking
RAG (Lewis et al, 2020)
ATLAS (lzacard et al, 2022)
Evi. Generator (Asai et al, 2022a)

Dialog
BlenderBot3 (Shuster et al.2022)
Internet-augmented generation (Komeili et a., 2022)

Summary
FLARE (Jiang et al, 2023)

Machine Translation
kNN-MT (Khandelwal et al., 2020)TRIME-MT (Zhong et al., 2022)

Code Generation
DocPrompting (Zhou et al., 2023
Natural ProverWelleck et al., 2022)

Natural Language Inference
kNN-Prompt (Shi et al., 2022)
NPM (Min et al., 2023)

Sentiment analysis
kNN-Prompt (Shi et al., 2022)NPM (Min et al., 2023)

Commonsense reasoning
Raco (Yu et al, 2022)

Naive RAG

Step1 Indexing

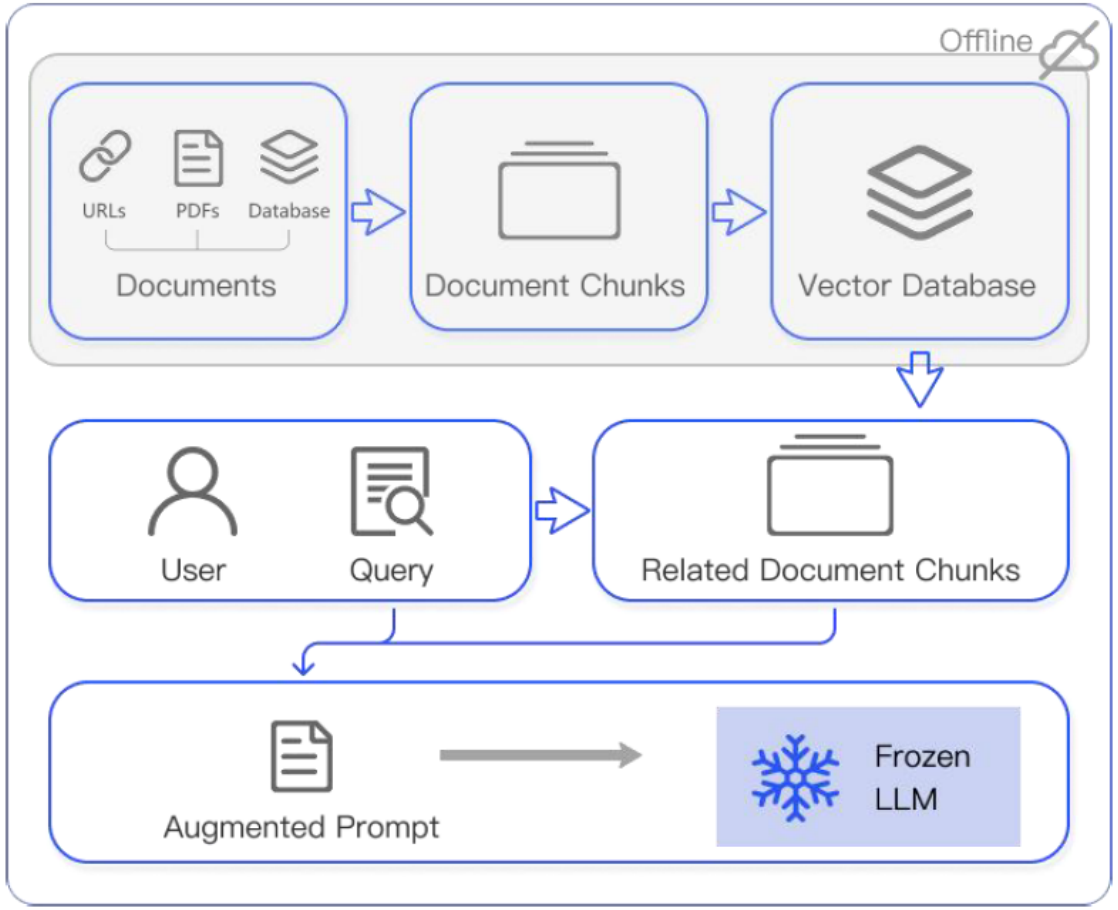
1. Divide the document into even chunks, each chunk being a piece of the original text.
2. Using the encoding model to generate an embedding for each chunck.
3. Store the Embedding of each block in the vector database.

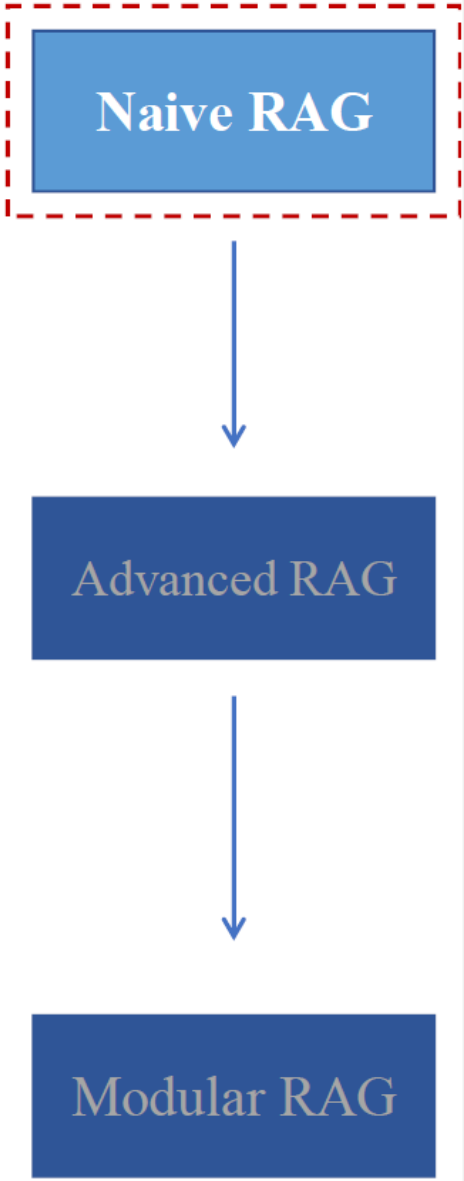
Step2 Retrival

Retrieve the k most relevant documents using vector similarity search.

Step3 Generation

The original query and the retrieved text are combined and input into a LLM to get the final answer

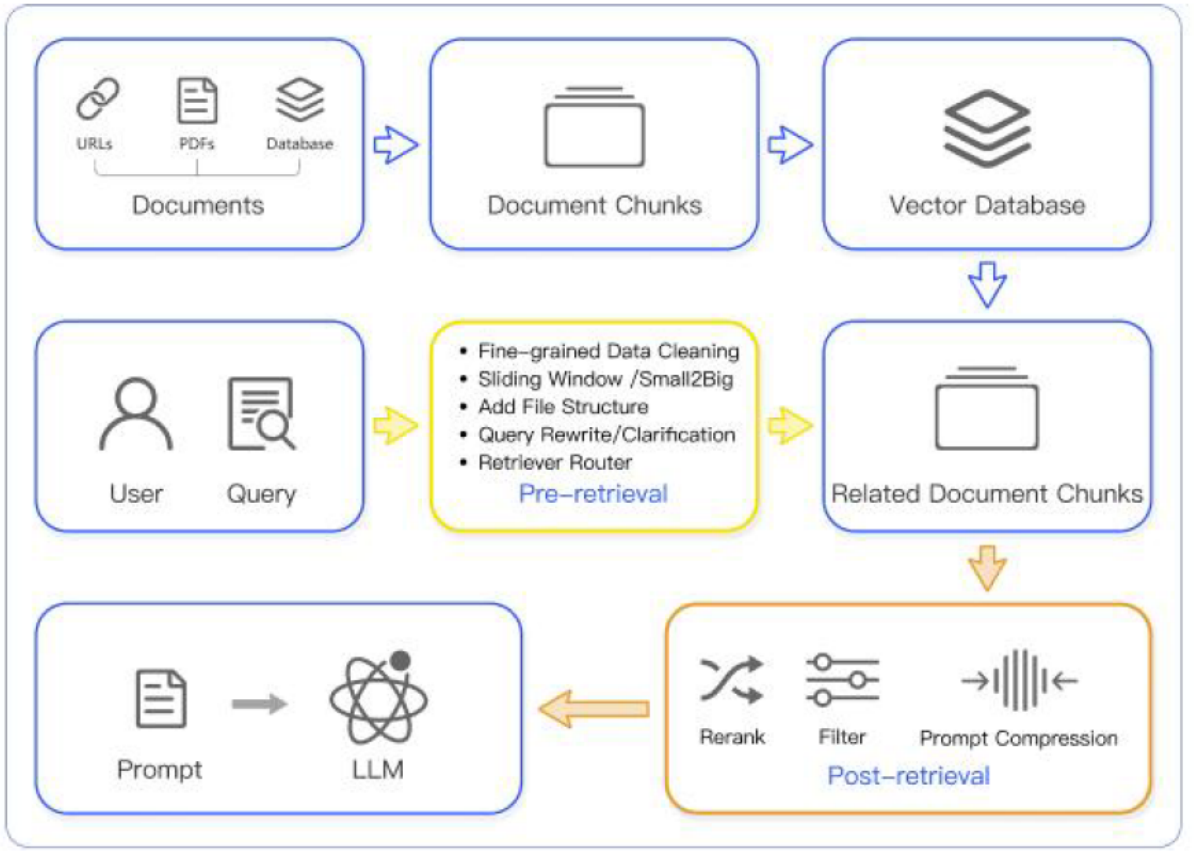


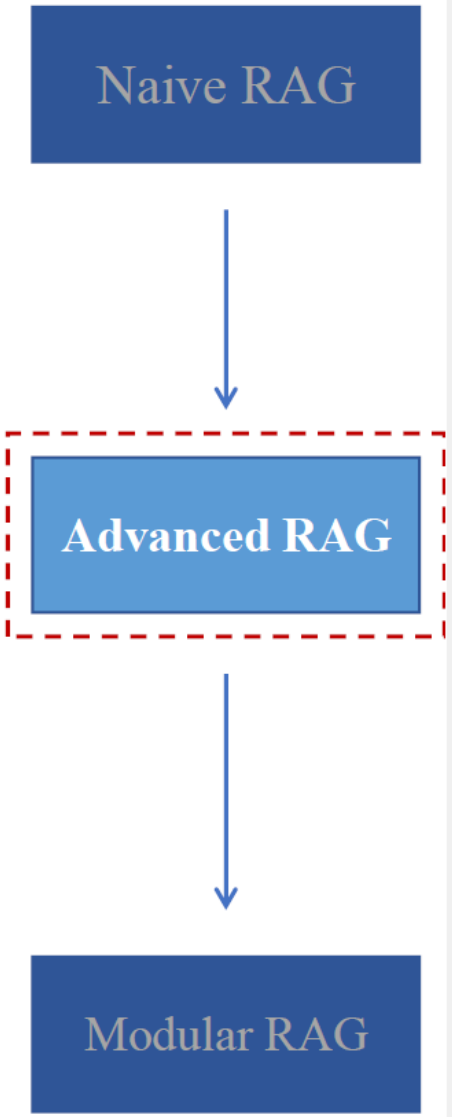


Advanced RAG

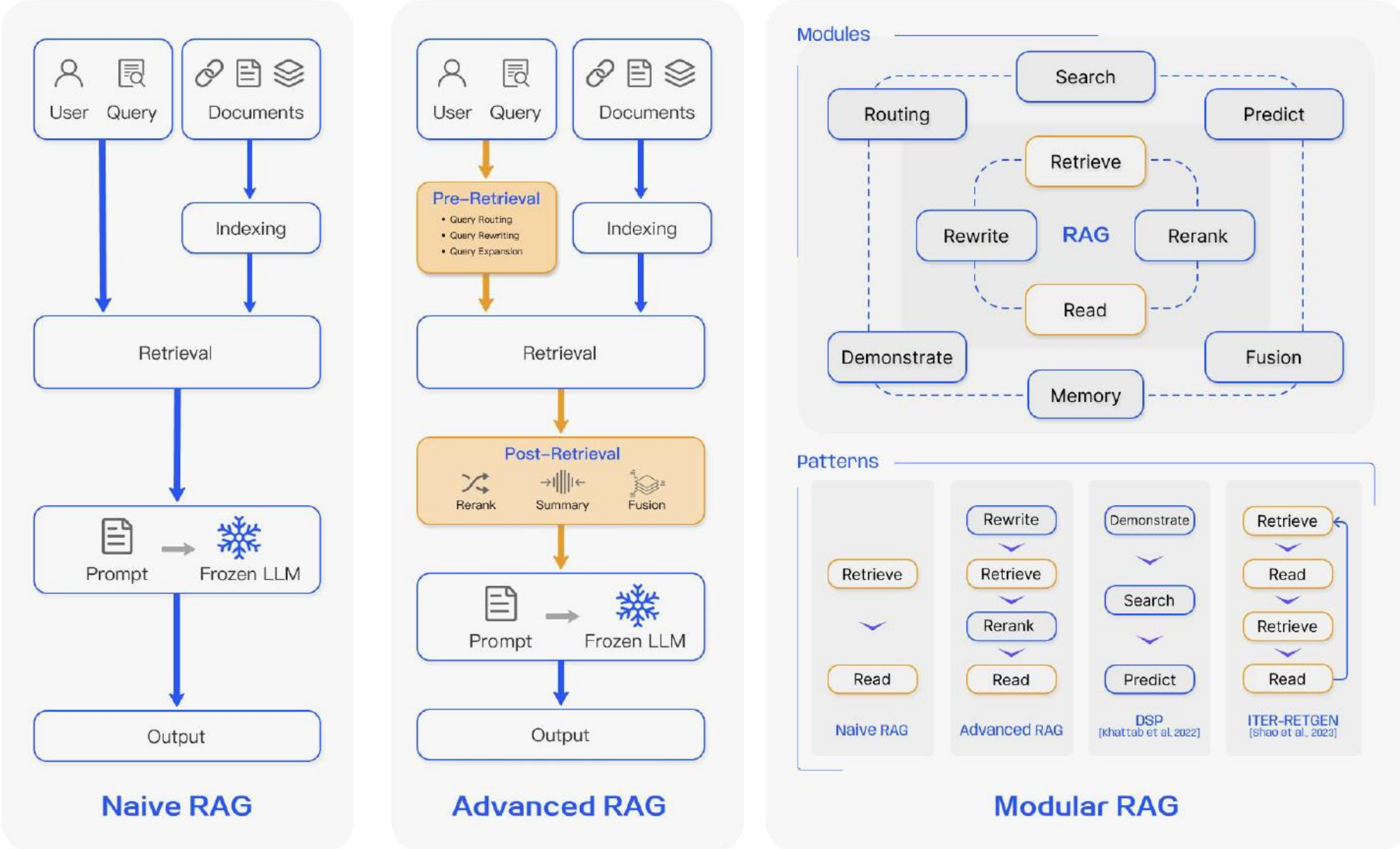
Index Optimization → Pre-Retrieval Process → Retrieval → Post-Retrieval Process → Genaration

- **Optimizing Data Indexing:** sliding window, fine-grained segmentation、adding metadata
- **Pre-Retrieval Process:** retrieve routes, summaries, rewriting, and confidence judgment
- **Post-Retrieval Process:** reorder, filter content retrieval





Comparison of RAG Paradigms



The three key questions of RAG

What to retrieve ?

- Token
- Phrase
- Chunk
- Paragraph
- Entity
- Knowledge graph

When to retrive ?

- Single search
- Each token
- Every N tokens
- Adaptive search

How to use the retrieved information ?

- Input/Data Layer
- Model/Intermediate Layer
- Output/Prediction Layer

Other Issues

Augmentation stage:
- Pre-training
- Fine-tuning
- Inference

Retrieval choice:
- BERT
- Roberta
- BGE
- .....

Model Collaboration



Scale selectionz

Generation choice:
- GPT
- Llama
- T5
- .....

Key issue of RAG — What to retrieve

coarse

Retrieval granularity

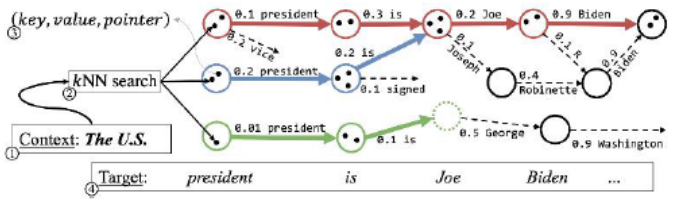
meticulous

low

level of structuration

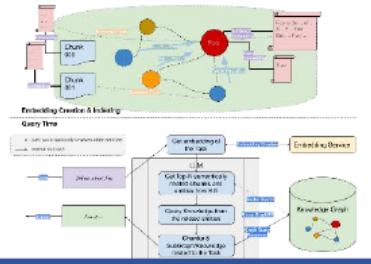
High

Chunk | In-Context RAG 2023



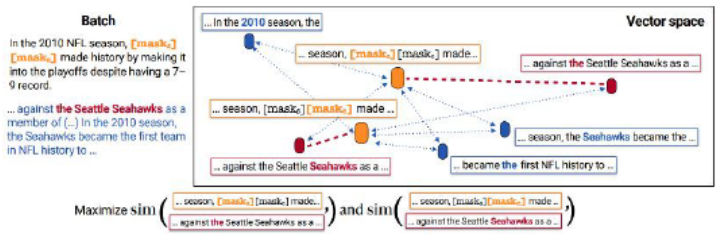
The search is broad, recalling a large amount of information, but with low accuracy, high coverage but includes much redundant information.

Knowledge Graph | 2023

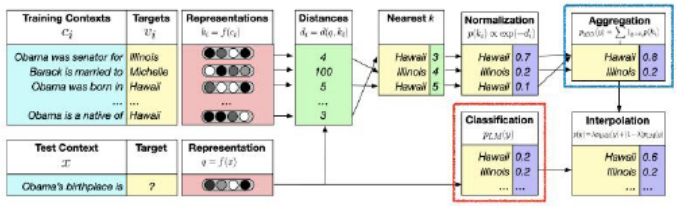


Richer semantic and structured information, but the retrieval efficiency is lower and is limited by the quality of KG.

Phrase | NPM 2023

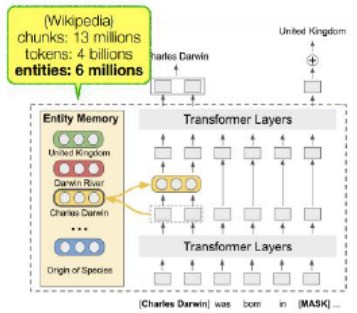


Token | KNN-LMM 2019



It excels in handling long-tail and cross-domain issues with high computational efficiency, but it requires significant storage.

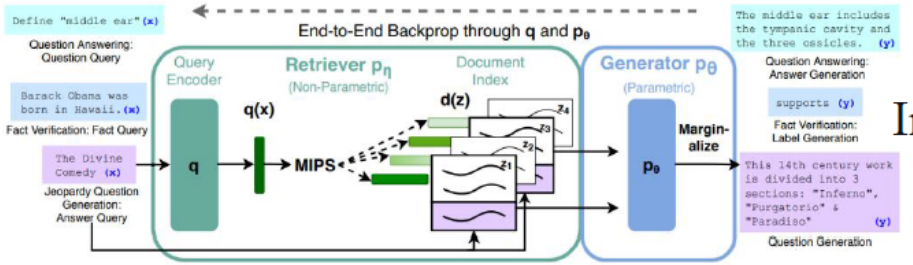
Entity | EasE 2022



Key issue of RAG — How to use the retrieved content

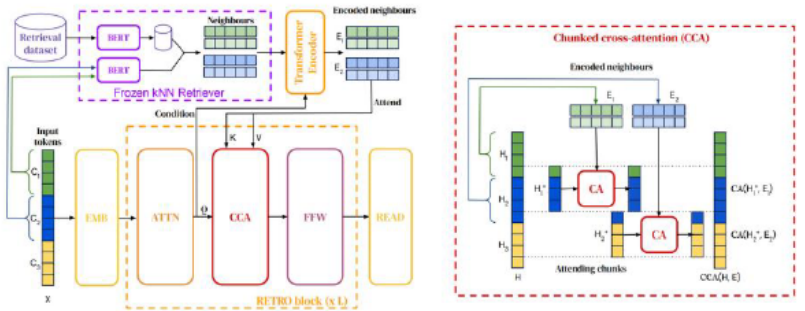
Integrating the retrieved information into different layers of the generation model,during inference process.

Integrate retrieval positions.



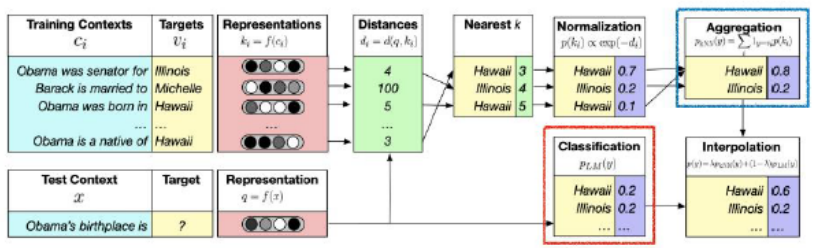
Input / Date layer

Using simple, but unable to support the retrieval of more knowledge blocks, and the optimization space is limited.



Model / Interlayer

Supports the retrieval of more knowledge blocks, but introduces additional complexity and must be trained.



Output /Prediction layer

Ensuring the output results are highly relevant to the retrieval content, but the efficiency is low.

Key issue of RAG — When to retrieve

High efficiency, but low relevance of the retrieved documents

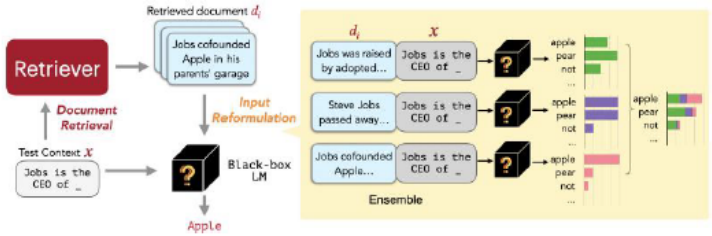
Balancing efficiency and information might not yield the optimal solution

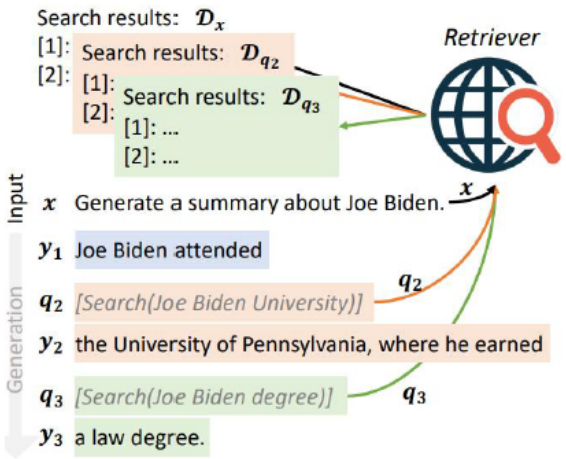
A large amount of information with low efficiency and redundant information.

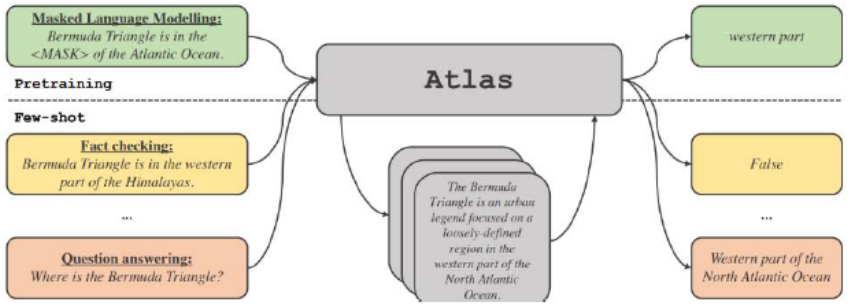
Once | Replug 2023

Adaptive | Flare 2023

Every N Tokens | Atlas 2023







Conducting once search during the reasoning process.

Adaptively conduct the search.

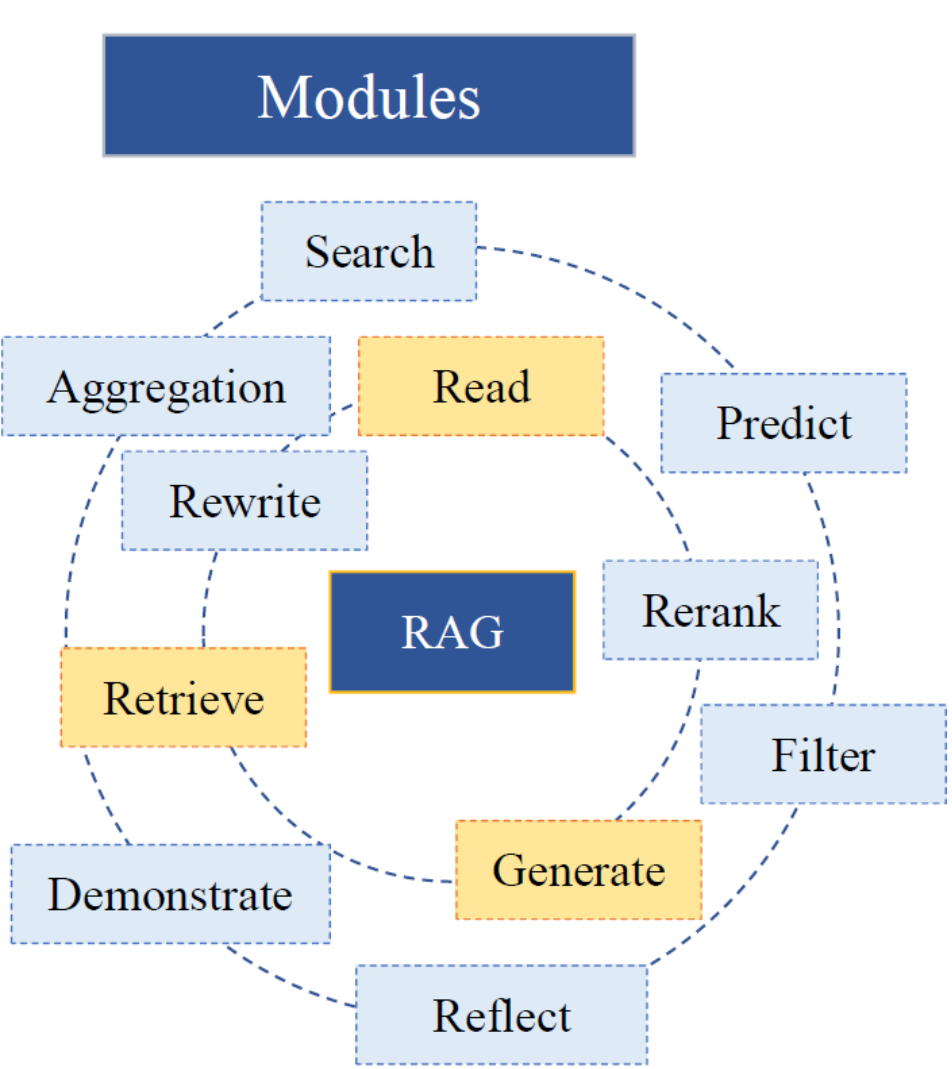
Retrieve once for every N tokens generated.

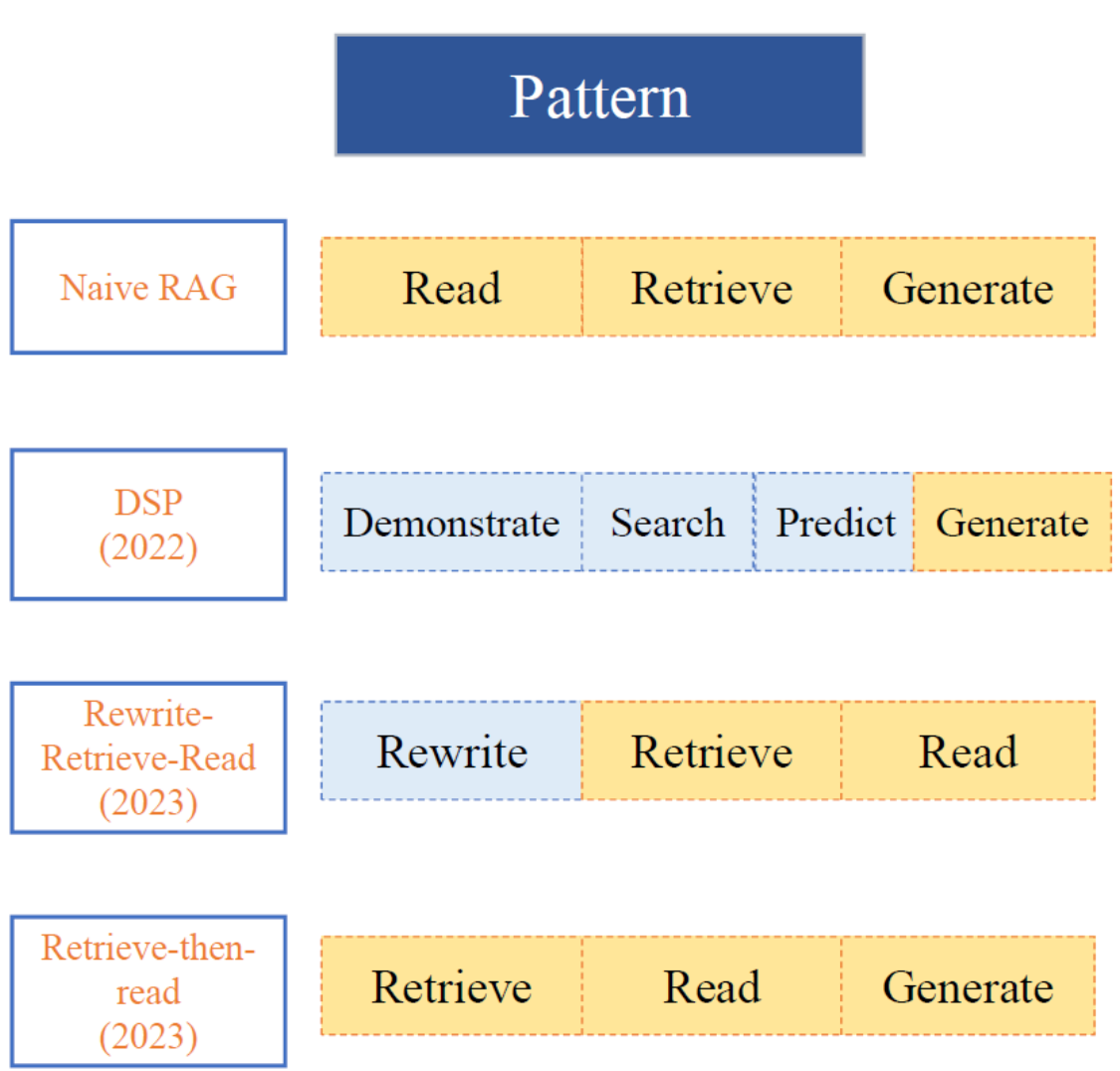
Low

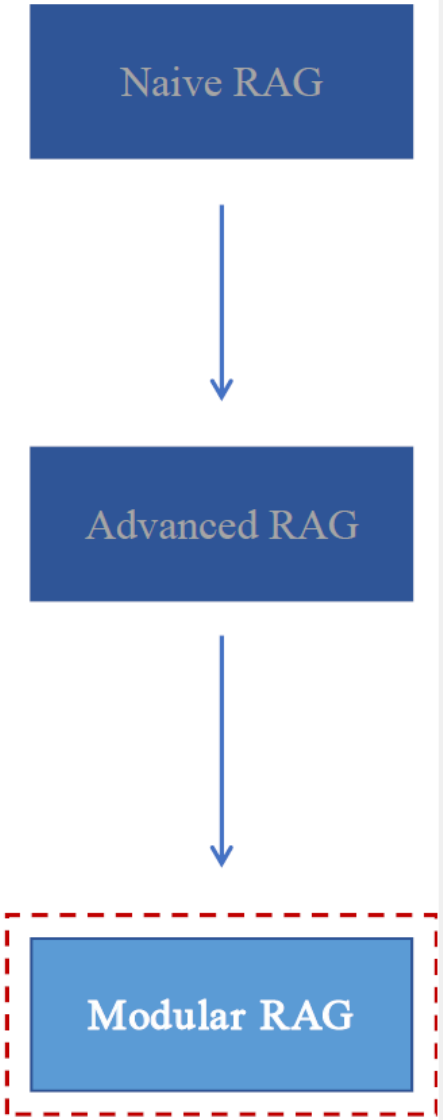
Retrieval frequency

High

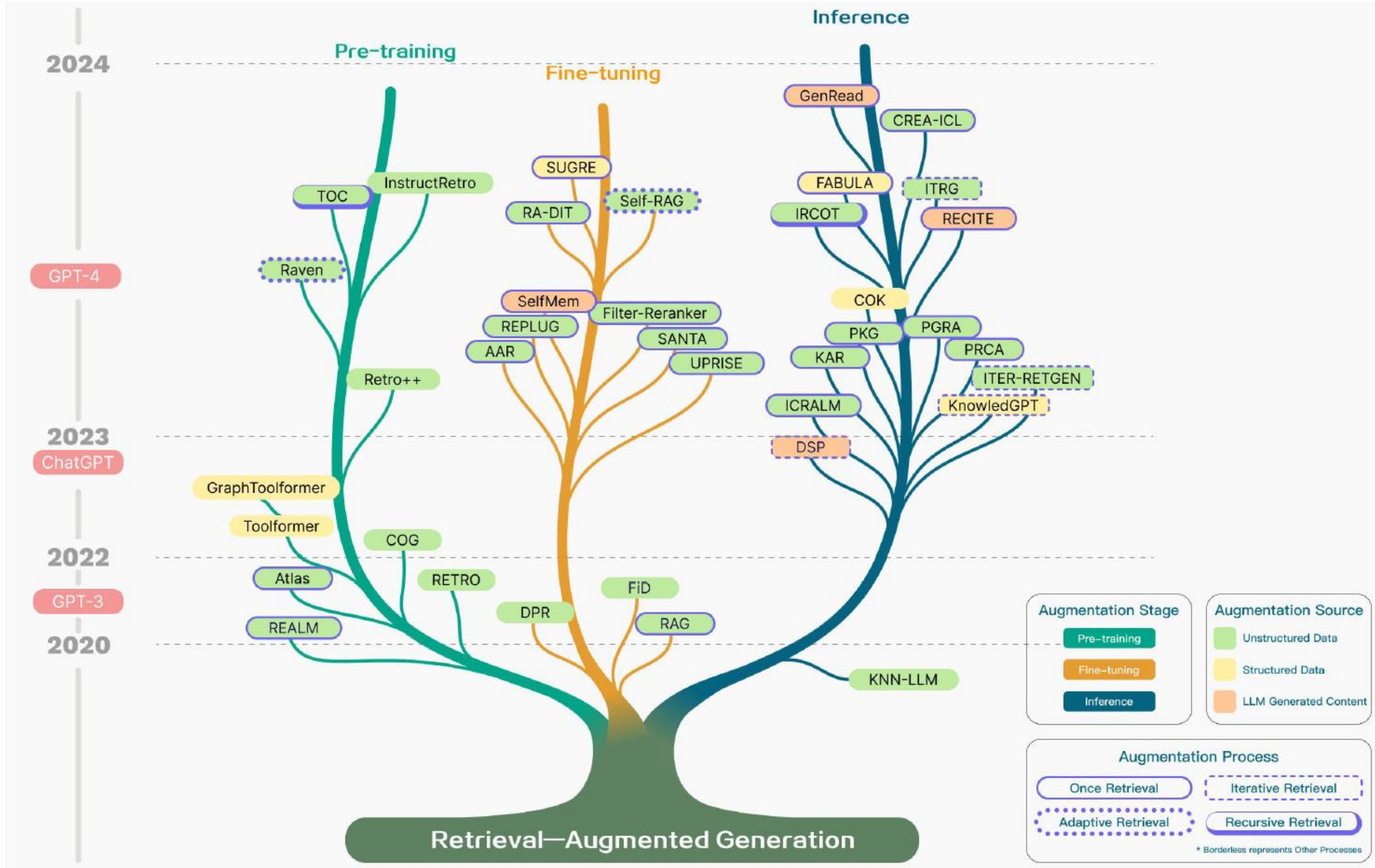
Modular RAG


Modules
Search
Aggregation
Read
Predict
Rewrite
RAG
Rerank
Retrieve
Filter
Generate
Demonstrate
Reflect


Pattern
Naive RAG
Read
Retrieve
Generate
DSP (2022)
Demonstrate
Search
Predict
Generate
Rewrite-Retrieve-Read (2023)
Rewrite
Retrieve
Read
Retrieve-then-read (2023)
Retrieve
Read
Generate


Naive RAG
Advanced RAG
Modular RAG

Overview of RAG Development



RAG Industry Application Practices

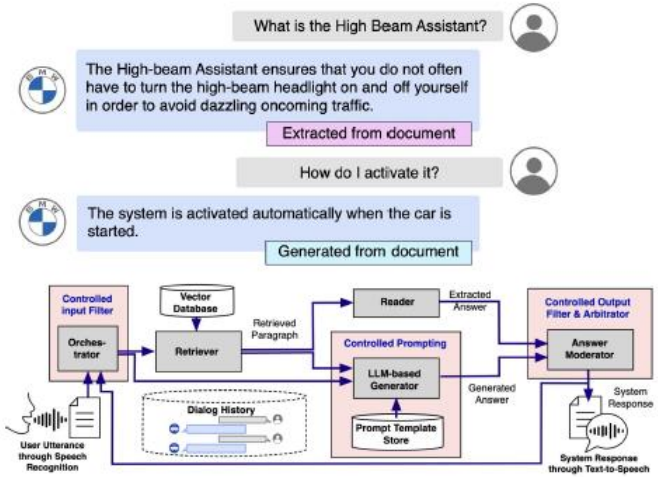


NetEase - ChatBI

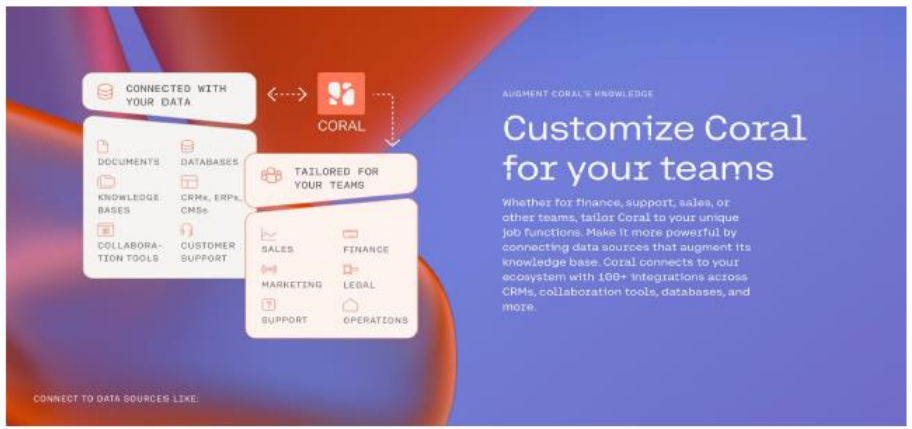
The intelligent upgrade of traditional industries

RAG

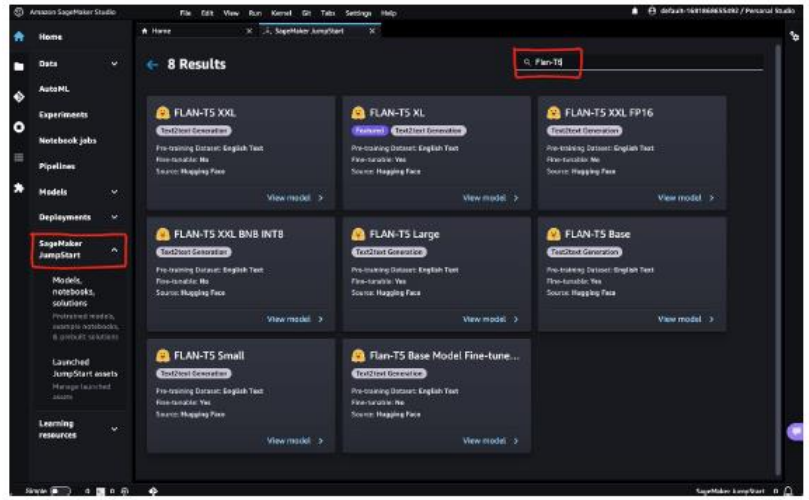
AI Toolchain Enhancement



BMW - CarExpert



Cohere - Coral



Amazon - Kendra

RAG $ On-Device ML

Using RAG with **on-device ML** for data-sensitive applications.

Example: Smart assistants that retrieve local data (e.g., notes, contacts) and generate responses without cloud dependency.



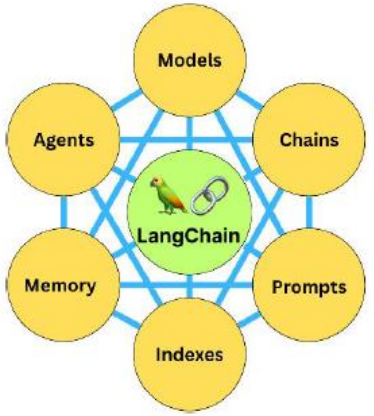


Chain Of Thought Prompting

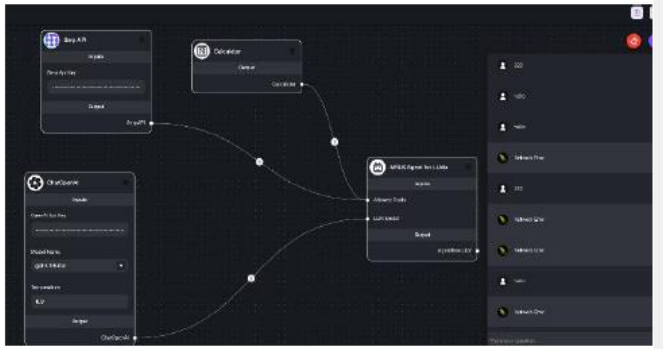
The journey of a thousand miles begins with a single step — and sometimes, that step is just breaking the problem into smaller steps.

Existing Tech Stack for RAG

Name	Pros	Cons
LangChain	Modular, full-featured	Inconsistent behavior ,API conceals details,complexity and low flexibility.
LlamaIndex	Focus on RAG	Requires combination use, low customization.
FlowiseAI	Easy to get started, visualized workflows.	Does not support complex scenarios.
AutoGen	Adapts to multi-agent scenarios.	Low efficiency, requires multiple rounds of dialogue.



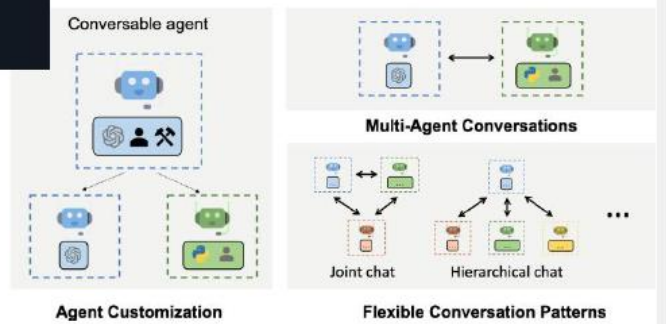
LangChain



FlowiseAI



LlamaIndex



AutoGen